

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a9ff8fbb57ea77769.jsonl`  
- SHA-256: `f281fda8d0241d4256ccc2f7b09086013a30f9ed2b46880cbc2f75ab8644933d`  
- Source modified: `2026-06-13T19:12:48+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_095035a0-e9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:09:22.948Z)

Review the new `PersonDetector.detections_per_frame` in `backend/pipeline/detectors/person_detector.py`. Verify: `cap.set(CAP_PROP_POS_FRAMES)+sequential-read` frame indexing is correct (the `frame_idx` labels vs actual decoded frames), `frame_skip` sampling, tracker fps, resource release (`cap.release` on all paths incl. early returns), and that the returned frame indices can actually align with WASB trajectory `.frame`. Note GPU-only caveats but find any LOGIC bug testable without a GPU.

Repo root = `C:/Users/avidu/Projects/badminton-highlight-indexer`. The change implements fusion P2 (`MULTI_SIGNAL_FUSION_PLAN.md`): a new default-OFF `FusionSegmenter` (`backend/pipeline/segmenters/fusion_hybrid.py`) that subclasses `TrajectoryHybridSegmenter`, persists shuttle net crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in `backend/pipeline/detectors/fusion_features.py`. NEW files: `backend/pipeline/detectors/fusion_features.py`, `backend/pipeline/segmenters/fusion_hybrid.py`. EDITED files (use ``git --no-pager diff -- <path>`` to see edits): `backend/pipeline/segmenters/trajectory_hybrid.py`, `backend/pipeline/detectors/person_detector.py`, `backend/config/models.py`, `backend/pipeline/segmenters/__init__.py`. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:09:29.397Z)

I'll review the new `PersonDetector.detections_per_frame` method and the surrounding fusion P2 changes. Let me start by reading the relevant files.

3. user (2026-06-13T19:09:30.419Z)

```
1      """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.  
2  
3      Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):  
4      the person cue is a swappable producer, not logic baked into one segmenter. Today  
5      HybridYoloSegmenter is the only consumer (it delegates Stage-1 here, byte-  
identically);  
6      next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and  
7      later the learned classifier reads its persisted features. Keeping detection here (not  
in  
8      the segmenter) is what makes "add a cue = register a producer" true.  
9  
10     Behaviour is unchanged from the in-lined version ? only the home moved:  
11     - lazy_load_model` ? load a permissively-licensed torchvision detector  
12     - make_tracker` ? a fresh supervision ByteTrack per chunk  
13     - detect_and_track` ? one frame ? confident persons with persistent IDs  
14     - extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows  
15     with the motion stats (peak/mean_velocity`, active_frame_count`, track_id_count`)
```

```

16         that feed the candidate `stats` JSON and the learned fusion features.
17
18     Heavy deps (torchvision, supervision) are imported lazily inside the methods so
importing
19     the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
20     the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
21     (ADR-005; ultralytics' AGPL YOLO was dropped).
22     """
23     import logging
24     from typing import Any, Dict, List, Optional, Tuple
25
26     import cv2
27     import torch
28
29     from backend.utils.geometry import get_velocity_normalised
30
31     logger = logging.getLogger(__name__)
32
33     # torchvision detection models are trained on COCO where the "person" category is
34     # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
35     # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
36     _PERSON_CLASS_ID = 1
37
38     # Permissively-licensed torchvision detectors selectable via config
39     # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
40     _DETECTOR_FACTORIES = {
41         "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
42         "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
43         "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
44     }
45     _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
46
47
48     class PersonDetector:
49         """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
50
51         Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
52         ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
53         min_action_window_duration}``); the model loads lazily on first use.
54         """
55
56         def __init__(self, config: Dict[str, Any]):
57             self.config = config
58             # Dynamic torchvision module (None until lazy_load_model); typed Any so the
59             # `self.model([tensor])` inference call type-checks without a quarantine.
60             self.model: Any = None
61             self.device: Optional[str] = None
62
63         def lazy_load_model(self) -> None:
64             if self.model is not None:
65                 return
66
67             # Imported lazily so the module imports without torchvision present and so
68             # the test suite (which pre-sets self.model) never pulls in model weights.
69             from torchvision.models import detection as tv_detection
70
71             idx_cfg = self.config.get("indexing", {})
72             use_gpu = idx_cfg.get("use_gpu", True)
73             self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
74             if use_gpu and not torch.cuda.is_available():
75                 logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
76
77             model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
78             if model_name not in _DETECTOR_FACTORIES:

```

```

79         logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
80         model_name = _DEFAULT_DETECTOR
81
82         builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
83         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
84         # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
85         self.model = builder(weights="DEFAULT")
86         self.model.eval()
87         self.model.to(self.device)
88
89     def make_tracker(self, fps: float) -> Any:
90         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
91
92         Imported lazily so tests can mock this method without supervision installed.
93         """
94         import supervision as sv
95         frame_rate = max(1, int(round(fps)))
96         return sv.ByteTrack(frame_rate=frame_rate)
97
98     def detect_and_track(self, frame: Any, tracker: Any,
99                         score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
100         """Detect persons in a frame and assign persistent track IDs.
101
102         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
103         This replaces the old ultralytics ``model.track(...)`` call, which bundled
104         detection AND association: torchvision does the detection, supervision's
105         ByteTrack does the association.
106         """
107         import supervision as sv
108
109         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
110         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
111         tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255.0).to(self.device)
112         with torch.no_grad():
113             outputs = self.model([tensor])
114
115         out = outputs[0]
116         boxes = out["boxes"].detach().cpu().numpy()
117         labels = out["labels"].detach().cpu().numpy()
118         scores = out["scores"].detach().cpu().numpy()
119
120         # Keep only confident person detections.
121         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
122         if not mask.any():
123             return []
124
125         # ByteTrack requires confidence to be populated (it filters on it internally).
126         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
127         tracked = tracker.update_with_detections(dets)
128
129         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
130         for i in range(len(tracked)):
131             tid = tracked.tracker_id[i]
132             if tid is None:
133                 continue
134             x1, y1, x2, y2 = tracked.xyxy[i]
135             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
136         return result
137
138     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
velocity_thresh: float, merge_gap: float,

```

```

140         frame_skip: int = 3,
141         chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
142     """
143     Runs person detection + tracking on a video chunk and extracts high-motion
144     action windows. Returns a list of dicts in the full video's timeframe, each
with:
145         start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
146         track_id_count, chunk_index.
147
148     Args:
149         frame_skip: Process every Nth frame (1 = every frame). Higher values
150             dramatically reduce inference cost with minimal quality loss.
151         chunk_index: Index of the source chunk (recorded on each window for
traceability).
152     """
153     cap = cv2.VideoCapture(chunk_path)
154     if not cap.isOpened():
155         logger.error(f"Could not open {chunk_path}")
156         return []
157
158     fps = cap.get(cv2.CAP_PROP_FPS)
159     if fps <= 0:
160         fps = 30.0
161
162     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
163     if frame_width <= 0:
164         frame_width = 1280.0 # Sensible fallback
165
166     # Effective sampling rate after frame-skipping
167     effective_fps = fps / max(frame_skip, 1)
168
169     # A fresh tracker per chunk ? track IDs only need to be consistent within a
170     # chunk (windows are computed per chunk, then merged across chunks by time).
171     tracker = self.make_tracker(effective_fps)
172     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
173
174     # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
175     # so windows can be enriched with motion stats for logging + fine-tuning.
176     active_frames = []
177     frame_idx = 0
178     track_history: Dict[int, Tuple[float, float, float, float]] = {}
179
180     while True:
181         ret, frame = cap.read()
182         if not ret:
183             break
184
185         # Skip frames for performance ? at 30fps, every 3rd frame still
186         # gives ~10 samples/second, plenty for human-scale velocity.
187         if frame_idx % max(frame_skip, 1) != 0:
188             frame_idx += 1
189             continue
190
191         # Detect persons + assign persistent track IDs.
192         detections = self.detect_and_track(frame, tracker, score_thresh)
193
194         frame_peak_velocity = 0.0
195         frame_active_ids = set()
196         current_ids = set()
197
198         for track_id, bbox in detections:
199             current_ids.add(track_id)
200

```

```

201         if track_id in track_history:
202             prev_bbox = track_history[track_id]
203             velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
204             if velocity > velocity_thresh:
205                 frame_peak_velocity = max(frame_peak_velocity, velocity)
206                 frame_active_ids.add(track_id)
207
208             track_history[track_id] = bbox
209
210             # Prune stale IDs that are no longer visible ? prevents false
211             # velocity spikes when the tracker reassigns an old ID.
212             track_history = {k: v for k, v in track_history.items() if k in current_ids}
213
214             if frame_active_ids:
215                 active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
216
217             frame_idx += 1
218
219         cap.release()
220
221         # Group active frames into continuous windows
222         if not active_frames:
223             return []
224
225         max_gap_frames = int(merge_gap * fps)
226
227         def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
228             velocities = [v for _, v, _ in group]
229             track_ids = set().union(*[ids for _, _, ids in group])
230             return {
231                 "start": group[0][0] / fps + chunk_start,
232                 "end": group[-1][0] / fps + chunk_start,
233                 "active_frame_count": len(group),
234                 "peak_velocity": max(velocities),
235                 "mean_velocity": sum(velocities) / len(velocities),
236                 "track_id_count": len(track_ids),
237                 "chunk_index": chunk_index,
238             }
239
240         windows = []
241         group = [active_frames[0]]
242         for af in active_frames[1:]:
243             if af[0] - group[-1][0] <= max_gap_frames:
244                 group.append(af)
245             else:
246                 windows.append(_finalize(group))
247                 group = [af]
248         windows.append(_finalize(group))
249
250         # Filter out extremely short action windows
251         min_window_duration = self.config.get("indexing",
{ }).get("min_action_window_duration", 2.0)
252         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
253
254         return windows
255
256     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
257                             frame_skip: int = 3) -> Dict[str, Any]:
258         """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
video.
259
260         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
261         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
discards the

```

```

262         per-frame boxes), this returns the raw per-frame detections so the fusion
features can
263         align them with the shuttle trajectory. It reads the source by **absolute frame
index**
264         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
match
265         the WASB trajectory's ``.frame`` exactly ? that alignment is what
266         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
267
268         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
269         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
seek
270         accuracy varies by codec (it lands near the requested keyframe then decodes
forward);
271         treat the indices as ±a few frames on real footage ? fine for window-level
features.
272         """
273         self.lazy_load_model()
274         cap = cv2.VideoCapture(video_path)
275         if not cap.isOpened():
276             logger.error(f"Could not open {video_path}")
277             return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
278
279         fps = cap.get(cv2.CAP_PROP_FPS)
280         if fps <= 0:
281             fps = 30.0
282         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
283         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
284         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
285         step = max(frame_skip, 1)
286         tracker = self.make_tracker(fps / step)
287
288         start_frame = max(0, int(start_frame))
289         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
290         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
291         frame_idx = start_frame
292         while frame_idx <= end_frame:
293             ret, frame = cap.read()
294             if not ret:
295                 break
296             if (frame_idx - start_frame) % step == 0:
297                 frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
298                 frame_idx += 1
299         cap.release()
300         return {"frames": frames, "fps": fps,
301                 "frame_width": float(frame_width), "frame_height": float(frame_height)}
302

```

4. user (2026-06-13T19:09:31.569Z)

```

diff --git a/backend/pipeline/detectors/person_detector.py
b/backend/pipeline/detectors/person_detector.py
index 38ec2b3..5b931a4 100644
--- a/backend/pipeline/detectors/person_detector.py
+++ b/backend/pipeline/detectors/person_detector.py
@@ -252,3 +252,50 @@ class PersonDetector:
     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

     return windows
+
+ def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
+                          frame_skip: int = 3) -> Dict[str, Any]:
+     """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL video.
+

```

```

+     Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
+     ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and discards the
+     per-frame boxes), this returns the raw per-frame detections so the fusion features can
+     align them with the shuttle trajectory. It reads the source by absolute frame index
+     (seek + sequential read), NOT a re-extracted clip, so the returned frame indices match
+     the WASB trajectory's ``.frame`` exactly ? that alignment is what
+     ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
+
+     Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
+     "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
+     accuracy varies by codec (it lands near the requested keyframe then decodes forward);
+     treat the indices as  $\pm$  a few frames on real footage ? fine for window-level features.
+     """
+     self.lazy_load_model()
+     cap = cv2.VideoCapture(video_path)
+     if not cap.isOpened():
+         logger.error(f"Could not open {video_path}")
+         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
+
+     fps = cap.get(cv2.CAP_PROP_FPS)
+     if fps <= 0:
+         fps = 30.0
+     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
+     frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
+     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
+     step = max(frame_skip, 1)
+     tracker = self.make_tracker(fps / step)
+
+     start_frame = max(0, int(start_frame))
+     frames: Dict[int, List[Tuple[float, float, float, float]]] = {}
+     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
+     frame_idx = start_frame
+     while frame_idx <= end_frame:
+         ret, frame = cap.read()
+         if not ret:
+             break
+         if (frame_idx - start_frame) % step == 0:
+             frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
+         frame_idx += 1
+     cap.release()
+     return {"frames": frames, "fps": fps,
+           "frame_width": float(frame_width), "frame_height": float(frame_height)}

```

5. assistant (2026-06-13T19:09:34.316Z)

Now let me read the related files to understand how `detections\_per\_frame` is consumed and how WASB trajectory `.frame` aligns.

6. user (2026-06-13T19:09:35.272Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN S3.2/S4).
2
3     NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6     module only turns those signals into a handful of scale/translation-invariant
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is ``(x1, y1, x2, y2)`` in pixels; ``(track_id, box)`` when motion
needs it.
12    - per_frame is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame

```

```

13     index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14     - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15     - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16     detection counts ? the player-count-only mode).
17     - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point
36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]
41             xj, yj = polygon[j]
42             # Does the horizontal ray at y cross edge (i, j)?
43             if (yi > y) != (yj > y):
44                 x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                 if x < x_cross:
46                     inside = not inside
47             j = i
48         return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None."""

```



```

71         if court_polygon is None:
72             return True
73         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                         frame_height: float,
78                         court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79         """Per-frame count of in-court persons (one entry per sampled frame)."""
80         return [
81             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
82                 court_polygon))
83             for _frame_idx, dets in sorted(per_frame.items())
84         ]
85
86     def _median(vals: Sequence[float]) -> float:
87         if not vals:
88             return 0.0
89         s = sorted(vals)
90         n = len(s)
91         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93
94     def players_in_court(counts: Sequence[int]) -> float:
95         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96         return _median(list(counts))
97
98
99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
101 flicker)."""
102         if not counts:
103             return 0.0
104         return sum(1 for c in counts if c >= min_players) / len(counts)
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                           frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []
114         for a, b in zip(frames, frames[1:]):
115             prev = {tid: box for tid, box in per_frame[a]}
116             for tid, box in per_frame[b]:
117                 if tid in prev:
118                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121         if not steps:
122             return 0.0
123         return sum(steps) / len(steps)
124
125
126     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                         frame_height: float, net: Optional[Net],
128                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?

```

```

0.0.
133     """
134     if net is None or not per_frame:
135         return 0.0
136     axis, pos = net
137     both = 0
138     for _frame_idx, dets in per_frame.items():
139         near, far = False, False
140         for _tid, box in dets:
141             if not person_in_court(box, frame_width, frame_height, court_polygon):
142                 continue
143             fx, fy = _foot_norm(box, frame_width, frame_height)
144             coord = fx if axis == "x" else fy
145             if coord < pos:
146                 near = True
147             else:
148                 far = True
149         if near and far:
150             both += 1
151     return both / len(per_frame)
152
153
154 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
155                    pad_frac: float) -> bool:
156     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
157     width/height (the 'adjacent / in-hand' tolerance)?""""
158     x1, y1, x2, y2 = box
159     px = pad_frac * frame_width
160     py = pad_frac * frame_height
161     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
162
163
164 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                               frame_width: float, frame_height: float,
166                               pad_frac: float = 0.03) -> float:
167     """Fraction of (visible shuttle frames that were also person-sampled) in which the
168     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
169     ~0**
170     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
171     """
172     if not per_frame or frame_width <= 0 or frame_height <= 0:
173         return 0.0
174     aligned = 0
175     colocated = 0
176     for p in shuttle_points or []:
177         if not getattr(p, "visible", False):
178             continue
179         dets = per_frame.get(int(p.frame))
180         if dets is None: # shuttle frame wasn't person-sampled ? can't judge
181             continue
182         aligned += 1
183         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
184               for _tid, box in dets):
185             colocated += 1
186     return colocated / aligned if aligned else 0.0
187
188 # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189 # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190 PERSON_FEATURE_NAMES = [
191     "players_in_court", "two_sided_motion", "mean_player_motion",
192     "in_court_ratio", "shuttle_player_colocation",
193 ]

```

```

194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                     frame_width: float, frame_height: float,
198                                     court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                     net: Optional[Net] = None,
200                                     min_players: int = 2) -> Dict[str, float]:
201     """Aggregate the per-window person features from per-frame detections + the in-
window
202     shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203     counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204     return {
205         "players_in_court": players_in_court(counts),
206         "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207         "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208         "in_court_ratio": in_court_ratio(counts, min_players),
209         "shuttle_player_colocation": shuttle_player_colocation(
210             per_frame, shuttle_points, frame_width, frame_height),
211     }
212

```

7. user (2026-06-13T19:09:35.817Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13    never imports torch / touches the GPU.
14    - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17    **Persisted candidates remain the full superset** regardless, so the offline
experiment
18    harness can re-score any variant.
19
20    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
23    segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24    experiment harness (EXPERIMENT_HARNESS.md), never silently.
25    """
26    from typing import Any, Dict, List, Optional, Tuple
27
28    from backend.pipeline.segmenters.base import segmenter_registry
29    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30    from backend.pipeline.detectors.base import DetectorRunner
31    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig

```

```

33     from backend.pipeline.detectors import rally_gate
34     from backend.pipeline.detectors import fusion_features as ff
35
36
37     class FusionSegmenter(TrajectoryHybridSegmenter):
38         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
39 B)."""
40
41         family = "fusion"
42         display_label = "Fusion"
43
44         def __init__(self, db: Any, config: Any):
45             super().__init__(db, config)
46             # Built lazily only if the person cue is enabled (keeps torch/GPU off the
47 default path).
48             self._person: Optional[Any] = None
49
50         @property
51         def name(self) -> str:
52             return "fusion_hybrid"
53
54         @property
55         def description(self) -> str:
56             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
57 rallies, "
58 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
59 and the "
60 "AI provider sets semantic boundaries.")
61
62         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
63 Any]]:
64             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
65             if substrate == "tracknet":
66                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
67                 return TrackNetRunner(cfg), {
68                     "weights_path": cfg.weights_path,
69                     "queue_length": cfg.queue_length,
70                     "fusion_substrate": "tracknet",
71                 }
72             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
73             return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
74 "wasb"}
75
76         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77 frame_width: float, video_path: str,
78 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
80 video_path, idx_cfg)
81             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
82 persisted
83             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
84 estimated
85             # over the whole trajectory by rally_gate (camera-agnostic).
86             gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
87 min_crossings=0)
88             stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
89
90             fcfg = idx_cfg.get("fusion", {})
91             if fcfg.get("cue_flags", {}).get("person", False):
92                 stats.update(self._person_features(cw, points, fps, frame_width, video_path,
93 fcfg))
94             return stats
95
96         def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
97 frame_width: float, video_path: str,

```

```

87         fcfg: Dict[str, Any]) -> Dict[str, float]:
88         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109    def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                           window_stats: List[Dict[str, Any]],
111                           idx_cfg: Dict[str, Any]) -> List[int]:
112        """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113        (0) keep every window ? identical to the substrate. Persisted candidates are
114        untouched (full superset)."""
115        fcfg = idx_cfg.get("fusion", {})
116        min_crossings = int(fcfg.get("min_crossings", 0))
117        min_players = int(fcfg.get("min_players", 0))
118        person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120        keep: List[int] = []
121        for i, st in enumerate(window_stats):
122            if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                continue
124            if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                continue
126            keep.append(i)
127        return keep
128
129
130    segmenter_registry.register(FusionSegmenter)
131

```

8. assistant (2026-06-13T19:09:39.038Z)

Now let me look at the `TrajectoryHybridSegmenter` base to understand the seams, the WASB runner's trajectory shape (`.frame`), and how the existing `extract\_action\_windows\_from\_chunk` works on clips vs absolute frames.

9. user (2026-06-13T19:09:40.015Z)

```

1        """Shared engine for trajectory-substrate hybrid segmenters.
2
3        tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4        standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5        now the single home for the whole pipeline shape:

```

```

6
7     healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8     -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter
29    from backend.utils.video import get_video_duration, extract_video_chunk
30    from backend.sports import sport_registry
31    from backend.providers import provider_registry
32    from backend.pipeline.detectors.base import DetectorRunner
33
34    logger = logging.getLogger(__name__)
35
36
37    def _fmt_ts(seconds: float) -> str:
38        seconds = max(0, int(seconds))
39        h, rem = divmod(seconds, 3600)
40        m, s = divmod(rem, 60)
41        return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44    class TrajectoryHybridSegmenter(BasePlaySegmenter):
45        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46        boundaries."""
47
48        #: config section under `indexing` (velocity_threshold lives there), the
49        #: output subdir for trajectories, and the metadata detector-tag prefix.
50        family: str = ""
51        #: human-readable label used in log lines.
52        display_label: str = ""
53
54        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55        Any]]:
56            """Return (runner, detector-specific detection_params extras)."""
57            raise NotImplementedError
58
59        @staticmethod
60        def _probe_fps_width(video_path: str) -> tuple:
61            """Return (fps, frame_width) for a video, with sensible fallbacks."""
62            cap = cv2.VideoCapture(video_path)
63            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65            cap.release()
66            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68        @staticmethod
69        def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70            """Provider-reported exchange count, else a duration-based estimate.

```

```

69
70     One canonical implementation ? the twins had drifted (wasb relied on
71     ``int(None)`` raising into the fallback; same result, by accident).
72     """
73     shot_count = segment.get("shuttle_exchange_count")
74     if shot_count is None:
75         return max(3, int(duration / 0.8))
76     try:
77         return int(shot_count)
78     except (ValueError, TypeError):
79         return max(3, int(duration / 0.8))
80
81     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
82     -----
83     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
84                                frame_width: float, video_path: str,
85                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
86         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
87         BASE
88         returns exactly the 4 motion keys, so the trajectory twins persist byte-
89         identical
90         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
91         features). ``points`` are the whole-video trajectory points
92         (TrajectoryPoint)."""
93         return {
94             "peak_velocity": cw["peak_velocity"],
95             "mean_velocity": cw["mean_velocity"],
96             "active_frame_count": cw["active_frame_count"],
97             "track_id_count": cw["track_id_count"],
98         }
99
100     def _select_for_handoff(self, windows: List[Dict[str, Any]],
101                           window_stats: List[Dict[str, Any]],
102                           idx_cfg: Dict[str, Any]) -> List[int]:
103         """Indices of the candidate windows that proceed to the AI handoff (and thus may
104         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
105         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
106         Persisted candidates are NOT affected ? they remain the full superset for
107         offline
108         re-scoring."""
109         return list(range(len(windows)))
110
111     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
112                     player_pool: Optional[List[str]] = None,
113                     max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
114         # override-ignore: the ABC declares the Result contract (Success|Failure)
115         # but every live segmenter still returns a bare list ? the documented
116         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
117         label = self.display_label
118         # --- Sport profile + AI provider wiring (identical across segmenters) ---
119         sport_name = self.config.get("sport", "badminton")
120         try:
121             sport_profile = sport_registry.get(sport_name)
122         except KeyError as e:
123             logger.error(str(e))
124             return []
125
126         idx_cfg = self.config.get("indexing", {})
127         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
128 "gemini"))
129         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
130 "gemini-2.5-flash"))
131
132         api_key_env_var = f"{provider_name.upper()}_API_KEY"
133         api_key = os.environ.get(api_key_env_var)

```

```

127         if not api_key:
128             logger.error(
129                 f"{api_key_env_var} environment variable is not set! "
130                 f"To run the {provider_name} handoff, set your API key. "
131                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132             )
133             return []
134         try:
135             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136         except KeyError as e:
137             logger.error(str(e))
138             return []
139
140         video_duration = get_video_duration(video_path)
141         if video_duration <= 0:
142             logger.error("Invalid video duration.")
143             return []
144         fps, frame_width = self._probe_fps_width(video_path)
145
146         # --- Runner config + windowing thresholds ---
147         runner, runner_params = self._build_runner(idx_cfg)
148
149         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154         log_candidates = idx_cfg.get("log_yolo_candidates", True)
155         store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157         detection_params = {
158             "detector": self.name,
159             "velocity_thresh": velocity_thresh,
160             "merge_gap": merge_gap,
161             "min_action_window_duration": min_window_duration,
162             **runner_params,
163         }
164
165         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166         ok, msg = runner.healthcheck()
167         if not ok:
168             logger.error("%s WSL environment not ready: %s", label, msg)
169             return []
170         logger.info("%s WSL env OK: %s", label, msg)
171
172         # --- Phase 2: trajectory -> candidate rally windows ---
173         # Trajectory detectors process the whole video in one pass (they carry
174         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175         t_start = time.time()
176         out_dir = os.path.join("output", self.family)
177         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178         if not csv_path:
179             logger.error("%s produced no trajectory; aborting.", label)
180             return []
181
182         points = runner.parse_trajectory_csv(csv_path)
183         logger.info("Parsed %d trajectory points (%d visible).",
184                     len(points), sum(1 for p in points if p.visible))
185
186         all_candidate_windows = runner.trajectory_to_action_windows(
187             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189             min_window_duration=min_window_duration, chunk_index=0,
190         )

```



```

191         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192                     label, time.time() - t_start, len(all_candidate_windows))
193
194         if log_candidates:
195             for cw in all_candidate_windows:
196                 logger.info(f"[{label} rally]
197                             {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
198                             f"({cw['end'] - cw['start']:.1f}s) |
199                             frames={cw['active_frame_count']} "
200                             f"peak_v={cw['peak_velocity']:.3f}
201                             mean_v={cw['mean_velocity']:.3f}")
202
203         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204         fusion
205         # segmenter adds net_crossings + person-cue features. Computed once, reused for
206         both
207         # persistence and the handoff gate below.
208         window_stats = [
209             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
210             idx_cfg)
211             for cw in all_candidate_windows
212         ]
213
214         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
215         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
216         if store_candidates:
217             for i, cw in enumerate(all_candidate_windows):
218                 candidate_ids[i] = self.db.add_candidate_segment(
219                     video_id, detector=self.name,
220                     start_time=cw["start"], end_time=cw["end"],
221                     chunk_index=cw["chunk_index"], confidence=min(1.0,
222                     cw["peak_velocity"]),
223                     stats=window_stats[i], detection_params=detection_params,
224                     )
225
226         if not all_candidate_windows:
227             return []
228
229         # --- Phase 3: AI provider handoff over padded candidate clips ---
230         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
231         Persisted
232         # candidates above stay the full superset ? only the served play_segments are
233         gated.
234         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
235         idx_cfg)
236
237         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
238         os.makedirs(handoff_dir, exist_ok=True)
239         logger.info("Phase 3: %s validation on %d candidate windows...",
240                     provider_name, len(handoff_indices))
241
242         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
243             w_start = max(0, window["start"] - handoff_padding)
244             w_end = min(video_duration, window["end"] + handoff_padding)
245             w_dur = w_end - w_start
246             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
247             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
248             reencode=True):
249                 return []
250             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
251             segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
252                 label=f"Handoff {wi+1}")
253
254             valid = []
255             for seg in segments:
256                 try:
257                     seg["start_time"] = float(seg["start_time"]) + w_start

```

```

245         seg["end_time"] = float(seg["end_time"]) + w_start
246         seg["_candidate_id"] = candidate_ids[wi]
247         valid.append(seg)
248     except (ValueError, TypeError, KeyError) as e:
249         logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
250     try:
251         os.remove(w_path)
252     except OSError:
253         pass
254     return valid
255
256     ai_segments: List[dict] = []
257     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                     for wi in handoff_indices]
260         for future in futures:
261             ai_segments.extend(future.result())
262     try:
263         os.rmdir(handoff_dir)
264     except OSError:
265         pass
266     logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268     # --- Phase 4: DB population + candidate linking ---
269     segments_data = []
270     for segment in ai_segments:
271         try:
272             start = float(segment["start_time"])
273             end = float(segment["end_time"])
274             dur = end - start
275             metadata = {
276                 "shot_count": self._shot_count_from(segment, dur),
277                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278                 "detector": f"{self.family}-hybrid-{model_name}",
279             }
280             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281             if segment_id:
282                 candidate_id = segment.get("_candidate_id")
283                 if candidate_id is not None:
284                     self.db.link_candidate_to_segment(candidate_id, segment_id)
285                 segments_data.append({
286                     "id": segment_id, "start_time": start, "end_time": end,
287                     "duration": dur, "metadata": metadata,
288                 })
289         except Exception as e:
290             logger.warning("Skipping invalid segment struct: %s", e)
291
292     return segments_data
293

```

10. user (2026-06-13T19:09:41.010Z)

warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be replaced by CRLF the next time Git touches it

```

diff --git a/backend/config/models.py b/backend/config/models.py
index bb56881..7f8191c 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -75,6 +75,39 @@ class WasbIndexConfig(BaseModel):

```

```
min_free_gb: float = 0.0
```

```
+class FusionConfig(BaseModel):
+    """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
+
+    Every default reproduces control behaviour: the fusion segmenter persists the
+    shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
+    true ? the person-cue features, but it does NOT gate the served handoff unless a
+    threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
+    is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-only);
+    supplied ? off-court persons (spectators / adjacent courts) are excluded.
+    """
+    # Which trajectory substrate seeds the windows (shuttle stays primary).
+    substrate: Literal["wasb", "tracknet"] = "wasb"
+    # Windowing velocity threshold for the fusion family (the engine reads
+    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
+    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
+    velocity_threshold: float = 0.02
+    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
+    # enabled ? so the default path never imports torch / touches the GPU.
+    cue_flags: dict[str, bool] = Field(default_factory=dict)
+    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
+    # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-
+    scoring).
+    min_crossings: int = Field(default=0, ge=0)
+    # Served Gate B: when the person cue is on, drop windows with median in-court players
+    # < this. 0 = OFF.
+    min_players: int = Field(default=0, ge=0)
+    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
+    court_polygon: Optional[list[list[float]]] = None
+    # Net line for the person two-sided-motion split (person features only ? the shuttle
+    # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
+    net_axis: Literal["x", "y"] = "y"
+    net_position: float = Field(default=0.5, ge=0.0, le=1.0)
+
+class IndexingConfig(BaseModel):
+    default_segmenter: str = "yolo_hybrid"
+    use_gpu: bool = True
@@ -106,6 +139,7 @@ class IndexingConfig(BaseModel):
+
+    tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
+    wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
+    fusion: FusionConfig = Field(default_factory=FusionConfig)
+
+    @field_validator("default_segmenter")
+    @classmethod
diff --git a/backend/pipeline/segmenters/__init__.py b/backend/pipeline/segmenters/__init__.py
index c003d1a..fc5de77 100644
--- a/backend/pipeline/segmenters/__init__.py
+++ b/backend/pipeline/segmenters/__init__.py
@@ -4,6 +4,7 @@ from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
+from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter

__all__ = [
    "segmenter_registry",
@@ -13,4 +14,5 @@ __all__ = [
    "HybridYoloSegmenter",
    "TrackNetHybridSegmenter",
    "WasbHybridSegmenter",
+    "FusionSegmenter",
```

```

]
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index b2847c9..149282e 100644
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
@@ -78,6 +78,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
    except (ValueError, TypeError):
        return max(3, int(duration / 0.8))

+
+ # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -----
+ def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
+                             frame_width: float, video_path: str,
+                             idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
+     """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
+     returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
+     rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
+     features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
+     return {
+         "peak_velocity": cw["peak_velocity"],
+         "mean_velocity": cw["mean_velocity"],
+         "active_frame_count": cw["active_frame_count"],
+         "track_id_count": cw["track_id_count"],
+     }
+
+ def _select_for_handoff(self, windows: List[Dict[str, Any]],
+                         window_stats: List[Dict[str, Any]],
+                         idx_cfg: Dict[str, Any]) -> List[int]:
+     """Indices of the candidate windows that proceed to the AI handoff (and thus may
+     become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
+     ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
+     Persisted candidates are NOT affected ? they remain the full superset for offline
+     re-scoring."""
+     return list(range(len(windows)))
+
+ def process_video(self, video_id: str, video_path: str, # type: ignore[override]
+                   player_pool: Optional[List[str]] = None,
+                   max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
@@ -172,31 +197,36 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
    f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
    f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

+
+ # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
+ # segmenter adds net_crossings + person-cue features. Computed once, reused for both
+ # persistence and the handoff gate below.
+ window_stats = [
+     self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
+     for cw in all_candidate_windows
+ ]
+
+ # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
+ candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
+ if store_candidates:
+     for i, cw in enumerate(all_candidate_windows):
+         stats = {
+             "peak_velocity": cw["peak_velocity"],
+             "mean_velocity": cw["mean_velocity"],
+             "active_frame_count": cw["active_frame_count"],
+             "track_id_count": cw["track_id_count"],
+         }
+         candidate_ids[i] = self.db.add_candidate_segment(
+             video_id, detector=self.name,
+             start_time=cw["start"], end_time=cw["end"],

```

```

        chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
-        stats=stats, detection_params=detection_params,
+        stats=window_stats[i], detection_params=detection_params,
    )

    if not all_candidate_windows:
        return []

    # --- Phase 3: AI provider handoff over padded candidate clips ---
+    # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
+    # candidates above stay the full superset ? only the served play_segments are gated.
+    handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
    handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
    os.makedirs(handoff_dir, exist_ok=True)
    logger.info("Phase 3: %s validation on %d candidate windows...",
-        provider_name, len(all_candidate_windows))
+        provider_name, len(handoff_indices))

    def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
        w_start = max(0, window["start"] - handoff_padding)
@@ -225,8 +255,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):

        ai_segments: List[dict] = []
        with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
-            futures = [executor.submit(process_candidate_window, wi, cw)
-                        for wi, cw in enumerate(all_candidate_windows)]
+            futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
+                        for wi in handoff_indices]
        for future in futures:
            ai_segments.extend(future.result())
    try:

```

11. assistant (2026-06-13T19:09:44.111Z)

Now let me look at the WASB runner's trajectory point shape (the `.frame` attribute) and how trajectory_to_action_windows` computes window start/end (to verify the start_f = int(cw["start"] * fps)` round-trip aligns).`

12. user (2026-06-13T19:09:45.074Z)

```

1      """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4          1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5          2. copies wasb_infer.py into the WASB repo's src/ dir,
6          3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7          4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9      So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10     with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11     """
12     import os
13     import shlex
14     import logging
15     import subprocess
16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote

```

```

23
24     logger = logging.getLogger(__name__)
25
26     # Windows path to the inference wrapper we stage into WSL.
27     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/WASB-SBDT"
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "wasb"
37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41         keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42         min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43
44         @classmethod
45         def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
46             w = (idx_cfg or {}).get("wasb", {}) or {}
47             return cls(
48                 distro=w.get("wsl_distro", "Ubuntu"),
49                 repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
50                 conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
51                 conda_env=w.get("conda_env", "wasb"),
52                 weights_path=w.get("weights_path",
53                                     "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
54                 sport=w.get("sport", "badminton"),
55                 wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
56                 timeout_sec=int(w.get("timeout_sec", 1800)),
57                 keep_frames=bool(w.get("keep_frames", False)),
58                 min_free_gb=float(w.get("min_free_gb", 0.0)),
59             )
60
61
62     class WasbRunner(WslCommandMixin, DetectorRunner):
63         def __init__(self, cfg: WasbConfig):
64             self.cfg = cfg
65
66         def healthcheck(self) -> Tuple[bool, str]:
67             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
68             cmd = (f"conda activate {self.cfg.conda_env} && "
69                 f"python -c \"import torch; print('torch', torch.__version__, "
70                 f"'cuda', torch.cuda.is_available())\"")
71             try:
72                 res = self._wsl_bash(cmd)
73             except FileNotFoundError:
74                 return False, "`wsl` not found ? Windows + WSL2 required."
75             except subprocess.TimeoutExpired:
76                 return False, "WSL healthcheck timed out."
77             out = (res.stdout or "").strip()
78             if res.returncode != 0:
79                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
80             return True, out
81
82         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:

```

```

83         src_mnt = to_wsl_mnt_path(video_win_path)
84         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
85         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
86         {wsl_tilde_quote(dest)} && echo OK"
87         try:
88             res = self._wsl_bash(cmd)
89             except subprocess.TimeoutExpired:
90                 logger.error("Staging video into WSL timed out.")
91                 return None
92             if res.returncode != 0 or "OK" not in (res.stdout or ""):
93                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
94 res.stdout).strip())
95                 return None
96             return dest
97
98     def _stage_infer_script(self) -> bool:
99         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
100         src_mnt = to_wsl_mnt_path(_INFER_WIN)
101         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
102 OK"
103         res = self._wsl_bash(cmd)
104         return res.returncode == 0 and "OK" in (res.stdout or "")
105
106     def run_predict(self, video_win_path: str, output_win_dir: str,
107                     video_id: Optional[str] = None) -> Optional[str]:
108         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
109
110         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
111         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
112         can never reuse each other's staged frames, resumable cache, or CSV.
113
114         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
115         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
116         # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
117         # the CSV landed inside WSL while Windows polled the repo-relative path.
118         output_win_dir = os.path.abspath(output_win_dir)
119         os.makedirs(output_win_dir, exist_ok=True)
120         # Normalize for cross-platform basename (tests use Windows paths on Linux).
121         video_win_path = str(video_win_path).replace("\\", "/")
122         base = os.path.basename(video_win_path)
123         stem, ext = os.path.splitext(base)
124         # Unique, content-derived key: same filename + different bytes -> different key.
125         key = f"{stem}_{video_id[:12]}" if video_id else stem
126         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
127         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
128
129         if not self._stage_infer_script():
130             logger.error("Could not stage wasb_infer.py into WSL.")
131             return None
132         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
133         if staged_video is None:
134             return None
135         frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
136
137         # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
138         # WSL stage filesystem is already low so the user can clean up before it fills.
139         if self.cfg.min_free_gb > 0:
140             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
141             if free is not None and free < self.cfg.min_free_gb:
142                 logger.warning(
143                     "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
144 extraction "
145                     "may fill the disk ? consider running tools/cleanup_caches.py.",
146                     free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

```

```

144         cmd = (
145             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
146             f"python wasb_infer.py "
147             f"--video {wsl_tilde_quote(staged_video)} "
148             f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
149             f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
150             f"--sport {shlex.quote(self.cfg.sport)} "
151             f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}")
152         )
153         logger.info("Running WASB inference on %s ...", base)
154         try:
155             res = self._wsl_bash(cmd)
156         except subprocess.TimeoutExpired:
157             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
158             return None
159         if res.returncode != 0:
160             logger.error("WASB inference failed:\n%s", (res.stderr or
161 res.stdout)[-2000:])
162             return None
163         if not os.path.exists(expected_csv_win):
164             logger.error("WASB finished but expected CSV not found at %s",
165 expected_csv_win)
166             return None
167         logger.info("WASB trajectory CSV: %s", expected_csv_win)
168
169         # Success ? the trajectory CSV is the durable output; the staged video copy and
170         # the (huge) decoded frame cache are now pure intermediates. Delete them unless
171         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
172         # so the frame cache + detector cache remain for a resumable re-run.
173         if not self.cfg.keep_frames:
174             logger.info("Cache hygiene: removing staged video + frame cache for %s "
175                 "(set indexing.wasb.keep_frames=true to retain).", key)
176             self._wsl_rm_rf([frames_dir, staged_video])
177         return expected_csv_win
178
179         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
180         agnostic).
181         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
182         trajectory_to_action_windows =
183         staticmethod(TrackNetRunner.trajectory_to_action_windows)

```

13. assistant (2026-06-13T19:09:48.151Z)

Now let me look at the `TrackNetRunner.parse_trajectory_csv`` and `trajectory_to_action_windows`` to understand the `.frame`` attribute and how window start/end times are computed (critical for the round-trip `int(cw["start"] * fps)`` alignment).

14. user (2026-06-13T19:09:48.759Z)

```

1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14         Nothing here imports TensorFlow; all model code stays in WSL.
15         """

```



```

16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = ""                        # WSL path to the .h5/.keras weights
(REQUIRED)
38         queue_length: int = 5
39         stage_in_wsl: bool = True                     # copy video into WSL fs first
(perf)
40         wsl_stage_dir: str = "~/clips"                # where staged videos/outputs live
in WSL
41         timeout_sec: int = 1800                      # 30 min; kills a hung call (0 = no
timeout)
42         keep_frames: bool = False                    # retain staged video after success
(debug only)
43         min_free_gb: float = 0.0                     # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p
72         p = p.replace("\\", "/")

```

```

73         if len(p) >= 2 and p[1] == ":":
74             drive = p[0].lower()
75             return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\n"
107             )
108         try:
109             res = self._wsl_bash(cmd)
110         except FileNotFoundError:
111             return False, "wsl` not found ? is this running on Windows with WSL2
112             installed?"
113         except subprocess.TimeoutExpired:
114             return False, "WSL healthcheck timed out."
115         out = (res.stdout or "").strip()
116         if res.returncode != 0:
117             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118         return True, out
119
120 # ----- inference
121
122 def run_predict(self, video_win_path: str, output_win_dir: str,
123                 video_id: Optional[str] = None) -> Optional[str]:
124     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125     None.
126
127     ``output_win_dir`` is a Windows directory (under the repo's output/) that
128     WSL writes into via its /mnt view, so the Windows process can read the CSV
129     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
130     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
131     a filename cannot collide on the output CSV.
132     """
133     if not self.cfg.weights_path:
134         logger.error(
135             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
136             "in config.json to the WSL path of your trained TrackNetV4 weights."
137         )

```

```

135         return None
136
137     # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
138     # relative paths through unchanged, so WSL would resolve them against the
139     # predict.py cwd instead of the repo (same bug class as wasb_runner).
140     output_win_dir = os.path.abspath(output_win_dir)
141     os.makedirs(output_win_dir, exist_ok=True)
142     # Normalize separators so basename/splitext work when tests (or collector)
143     # pass Windows-style paths on a Linux host.
144     video_win_path = str(video_win_path).replace("\\", "/")
145     base = os.path.basename(video_win_path)
146     stem, ext = os.path.splitext(base)
147
148     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149     if ext.lower() not in (".mp4", ".avi"):
150         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151         return None
152
153     wsl_out = to_wsl_mnt_path(output_win_dir)
154     # Namespace by video_id so two same-filename videos don't collide on the CSV.
155     # predict.py derives its output name from the (staged) video stem, so staging
156     # under the namespaced name propagates into "<key>_predict.csv".
157     key = f"{stem}__{video_id[:12]}" if video_id else stem
158     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160     # Resolve the video path WSL will read.
161     if self.cfg.stage_in_wsl:
162         staged = self._stage_video(video_win_path, f"{key}{ext}")
163         if staged is None:
164             return None
165         wsl_video = staged
166     else:
167         # No staging: predict.py reads /mnt directly and writes
168         # "<stem>_predict.csv".
169         # We cannot rename its output, so fall back to the un-namespaced name.
170         wsl_video = to_wsl_mnt_path(video_win_path)
171         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173     if self.cfg.min_free_gb > 0:
174         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175         if free is not None and free < self.cfg.min_free_gb:
176             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177                             "consider running tools/cleanup_caches.py.",
178                             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180     cmd = (
181         f"conda activate {self.cfg.conda_env} && "
182         f"cd {self.cfg.repo_dir}/src && "
183         f"python predict.py "
184         f"--video_path {wsl_tilde_quote(wsl_video)} "
185         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
186         f"--output_dir {wsl_tilde_quote(wsl_out)} "
187         f"--queue_length {self.cfg.queue_length}"
188     )
189     logger.info("Running TrackNet inference on %s ...", base)
190     try:
191         res = self._wsl_bash(cmd)
192     except subprocess.TimeoutExpired:
193         logger.error("TrackNet inference timed out after %ss.",
194                     self.cfg.timeout_sec)
195         return None
196
197     if res.returncode != 0:
198         logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
199                     res.stdout)[-2000:])

```

```

197         return None
198
199     if not os.path.exists(expected_csv_win):
200         logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201         return None
202         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205         # regenerable intermediate). On earlier failure we return above without
cleaning.
206         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207             logger.info("Cache hygiene: removing staged video for %s "
208                 "(set indexing.tracknet.keep_frames=true to retain).", key)
209             self._wsl_rm_rf([wsl_video])
210             return expected_csv_win
211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         dest = f"{self.cfg.wsl_stage_dir}/{base}"
219         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
220         try:
221             res = self._wsl_bash(cmd)
222         except subprocess.TimeoutExpired:
223             logger.error("Staging video into WSL timed out.")
224             return None
225         if res.returncode != 0 or "OK" not in (res.stdout or ""):
226             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
227             return None
228             return dest
229
230     # ----- CSV -> windows
231
232     @staticmethod
233     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
234         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
235         points: List[TrajectoryPoint] = []
236         with open(csv_win_path, newline="") as f:
237             reader = csv.DictReader(f)
238             for row in reader:
239                 try:
240                     points.append(TrajectoryPoint(
241                         frame=int(row["Frame"]),
242                         visible=int(row["Visibility"]) == 1,
243                         x=float(row["X"]),
244                         y=float(row["Y"]),
245                     ))
246                 except (KeyError, ValueError):
247                     continue
248             points.sort(key=lambda p: p.frame)
249             return points
250
251     @staticmethod
252     def trajectory_to_action_windows(
253         points: List[TrajectoryPoint],
254         fps: float,
255         frame_width: float,
256         chunk_start: float = 0.0,
257         velocity_thresh: float = 0.02,

```

```

258         merge_gap: float = 2.0,
259         min_window_duration: float = 2.0,
260         chunk_index: Optional[int] = None,
261     ) -> List[Dict[str, Any]]:
262         """Convert a shuttle trajectory into candidate rally windows.
263
264         A frame is "active" when the shuttle is visible AND its inter-frame
265         displacement (normalised by frame width, per second) exceeds
266         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
267         ``merge_gap`` seconds of each other are grouped into one window; short
268         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
269         shape feeding the AI-handoff phase is identical.
270         """
271         if fps <= 0:
272             fps = 30.0
273         if frame_width <= 0:
274             frame_width = 1280.0
275
276         # Compute per-frame normalised velocity from the last *visible* point.
277         active = [] # (frame_idx, velocity)
278         prev: Optional[TrajectoryPoint] = None
279         for p in points:
280             if not p.visible:
281                 prev = None # break the trajectory; absent shuttle = not in flight
282                 continue
283             if prev is not None:
284                 dframes = p.frame - prev.frame
285                 if dframes > 0:
286                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
287                     velocity = (dist / frame_width) * (fps / dframes)
288                     if velocity > velocity_thresh:
289                         active.append((p.frame, velocity))
290             prev = p
291
292         if not active:
293             return []
294
295         max_gap_frames = int(merge_gap * fps)
296
297         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
298             vels = [v for _, v in group]
299             return {
300                 "start": group[0][0] / fps + chunk_start,
301                 "end": group[-1][0] / fps + chunk_start,
302                 "active_frame_count": len(group),
303                 "peak_velocity": max(vels),
304                 "mean_velocity": sum(vels) / len(vels),
305                 "track_id_count": 1, # single shuttle; kept for window-shape parity
306                 "chunk_index": chunk_index,
307             }
308
309         windows = []
310         group = [active[0]]
311         for af in active[1:]:
312             if af[0] - group[-1][0] <= max_gap_frames:
313                 group.append(af)
314             else:
315                 windows.append(_finalize(group))
316                 group = [af]
317         windows.append(_finalize(group))
318
319         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
320

```

Now I have the full picture. Let me verify a few specific concerns. First, the round-trip alignment: windows are created with ``start = group[0][0] / fps + chunk_start`` where ``chunk_start=0.0``, so ``cw["start"] = frame / fps``. Then fusion does ``start_f = int(cw["start"] * fps) = int(frame)``. Let me check the float round-trip behavior precisely, and check what ``extract_action_windows_from_chunk`` does (the original) versus the new method regarding the frame-skip indexing semantics.

Let me check the existing tests to understand the contract and whether there's a test for ``detections_per_frame``.

16. user (2026-06-13T19:09:55.332Z)

warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'tests/test_trajectory_hybrid_equivalence.py', LF will be replaced by CRLF the next time Git touches it

```
backend/config/models.py | 34 ++++++
backend/pipeline/detectors/person_detector.py | 47 ++++++
backend/pipeline/segmenters/__init__.py | 2 +
backend/pipeline/segmenters/trajectory_hybrid.py | 50 ++++++-----
docs/README.md | 3 +-
tests/test_config.py | 30 ++++++
tests/test_trajectory_hybrid_equivalence.py | 22 ++++++
7 files changed, 177 insertions(+), 11 deletions(-)
---UNTRACKED TESTS---
M tests/test_config.py
M tests/test_trajectory_hybrid_equivalence.py
?? tests/test_fusion_features.py
?? tests/test_fusion_hybrid.py
```

17. user (2026-06-13T19:09:55.827Z)

Found 11 files
tests\test_fusion_hybrid.py
tests\test_fusion_features.py
backend\pipeline\segmenters__init__.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\detectors\person_detector.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\detectors\fusion_features.py
docs\QUALITY_ITERATIONS.md
docs\NEXT_STEPS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\EXPERIMENT_HARNESS.md

18. user (2026-06-13T19:09:59.133Z)

```
1      """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3      Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4      keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5      substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6      persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7      """
8      from types import SimpleNamespace
9      from unittest.mock import MagicMock, patch
10
11      from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12      from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14      # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
```

```

15     # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16     WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17             "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18     WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19             "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22     def _points():
23         pts = []
24         for f in range(30, 120):             # window A frames: y flips 100<->900 around the net
25             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
26             else 900.0))
27         for f in range(300, 360):             # window B frames: y == net level -> deadband -> no
28             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
29         return pts
30
31     def _runner():
32         r = MagicMock()
33         r.healthcheck.return_value = (True, "env ok")
34         r.run_predict.return_value = "out.csv"
35         r.parse_trajectory_csv.return_value = _points()
36         r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
37         return r
38
39
40     def _run(indexing=None, provider_segments=None):
41         provider = MagicMock()
42         provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
43         "end_time": 2.0}], {})
44         db = MagicMock()
45         db.add_candidate_segment.side_effect = [101, 102, 103, 104]
46         db.add_play_segment.return_value = 200
47         with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
48         {"weights_path": "w"})), \
49             patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)), \
50             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
51             return_value=30.0), \
52             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
53             return_value=True), \
54             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
55             preg, \
56             patch("os.environ.get", return_value="dummy_key"):
57             preg.get.return_value = provider
58             seg = FusionSegmenter(db, {"indexing": indexing or {}})
59             res = seg.process_video("v1", "clip.mp4")
60             return db, provider, res
61
62
63     def _stats_calls(db):
64         """Persisted stats dicts, in call order."""
65         return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
66
67
68     def test_net_crossings_persisted_and_person_off_by_default():
69         db, _provider, _res = _run()
70         stats = _stats_calls(db)
71         assert len(stats) == 2                                     # both windows persisted
72         (superset)
73         assert all("net_crossings" in s for s in stats)
74         # Window A (rally) crosses many times; window B (held) zero.

```

```

70         assert stats[0]["net_crossings"] >= 2
71         assert stats[1]["net_crossings"] == 0.0
72         # Person cue OFF by default -> NONE of the person features present.
73         assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
74         # The 4 base motion keys are still there.
75         assert all({"peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count"} <= set(s)
76                     for s in stats)
77
78
79     def test_persisted_candidates_are_superset_of_substrate_windows():
80         db, _provider, _res = _run()
81         spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
db.add_candidate_segment.call_args_list]
82         assert spans == [(1.0, 4.0), (10.0, 12.0)]          # exactly the substrate windows,
unfiltered
83
84
85     def test_default_hands_off_all_windows():
86         _db, provider, _res = _run()                        # min_crossings default 0 -> no
gating
87         assert provider.analyze_video.call_count == 2
88
89
90     def test_gate_a_drops_low_crossing_window_from_handoff_only():
91         db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
92         # Persistence is still the full superset (offline substrate intact)...
93         assert len(_stats_calls(db)) == 2
94         # ...but only the high-crossing window A reaches the AI handoff.
95         assert provider.analyze_video.call_count == 1
96
97
98     def test_person_cue_adds_features_when_enabled():
99         fake_pd = MagicMock()
100         fake_pd.detections_per_frame.return_value = {
101             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},
102             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
103         }
104         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
return_value=fake_pd):
105             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
106             stats = _stats_calls(db)
107             assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
108             assert all("net_crossings" in s for s in stats)
109             fake_pd.detections_per_frame.assert_called()      # the person detector WAS used
110

```

19. user (2026-06-13T19:09:59.291Z)

```

1     """Pure-logic unit tests for the person-cue fusion features
(backend.pipeline.detectors.fusion_features).
2
3     No GPU/torch/cv2 ? synthetic person boxes + shuttle points exercise every feature path
and
4     edge case. This is the testable core that proves the fusion math correct before any GPU
run.
5     """
6     from types import SimpleNamespace
7
8     import pytest
9
10    from backend.pipeline.detectors import fusion_features as ff
11
12
13    def _shuttle(frame, x, y, visible=True):

```



```

14         return SimpleNamespace(frame=frame, x=x, y=y, visible=visible)
15
16
17     # Square unit polygon and a left-half polygon (x in [0, 0.5]).
18     FULL = [(0.0, 0.0), (1.0, 0.0), (1.0, 1.0), (0.0, 1.0)]
19     LEFT_HALF = [(0.0, 0.0), (0.5, 0.0), (0.5, 1.0), (0.0, 1.0)]
20
21
22     # ----- point_in_polygon
23
24     def test_point_in_polygon_inside_outside():
25         assert ff.point_in_polygon((0.5, 0.5), FULL) is True
26         assert ff.point_in_polygon((1.5, 0.5), FULL) is False
27         assert ff.point_in_polygon((0.2, 0.5), LEFT_HALF) is True
28         assert ff.point_in_polygon((0.8, 0.5), LEFT_HALF) is False
29
30
31     def test_point_in_polygon_degenerate():
32         assert ff.point_in_polygon((0.5, 0.5), None) is False
33         assert ff.point_in_polygon((0.5, 0.5), [(0.0, 0.0), (1.0, 1.0)]) is False # <3
verts
34
35
36     # ----- person_in_court
37
38     def test_person_in_court_uses_foot_point_and_polygon():
39         # box foot = ((x1+x2)/2/W, y2/H). W=H=100.
40         in_box = (10, 10, 30, 30) # foot (0.2, 0.3) ? left half
41         out_box = (60, 10, 80, 30) # foot (0.7, 0.3) ? right half
42         assert ff.person_in_court(in_box, 100, 100, LEFT_HALF) is True
43         assert ff.person_in_court(out_box, 100, 100, LEFT_HALF) is False
44         # None polygon => everyone counts
45         assert ff.person_in_court(out_box, 100, 100, None) is True
46         # Unusable frame dims => foot (0,0); with a polygon that excludes origin => False
47         assert ff.person_in_court(in_box, 0, 0, LEFT_HALF) is True # (0,0) is on left half
edge-inside
48
49
50     # ----- counts / ratios
51
52     def test_in_court_counts_and_players_and_ratio():
53         near = (10, 10, 30, 30) # foot (0.2,0.3) in LEFT_HALF
54         far = (60, 10, 80, 30) # foot (0.7,0.3) NOT in LEFT_HALF
55         per_frame = {0: [(1, near), (2, far)], 1: [(1, near)], 2: [], 3: [(1, near), (2,
near)]}]
56
57         # With LEFT_HALF mask: counts = [1, 1, 0, 2]
58         counts = ff.in_court_counts(per_frame, 100, 100, LEFT_HALF)
59         assert counts == [1, 1, 0, 2]
60         assert ff.players_in_court(counts) == 1.0 # median of [0,1,1,2]
61         assert ff.in_court_ratio(counts, min_players=2) == pytest.approx(0.25)
62         assert ff.in_court_ratio(counts, min_players=1) == pytest.approx(0.75)
63         # No mask: far counts too -> [2,1,0,2]
64         assert ff.in_court_counts(per_frame, 100, 100, None) == [2, 1, 0, 2]
65         assert ff.players_in_court([]) == 0.0
66         assert ff.in_court_ratio([]) == 0.0
67
68
69     # ----- mean_player_motion
70
71     def test_mean_player_motion():
72         # track 1 moves 10px in x between frames; W=H=100 -> normalized disp 0.10.
73         per_frame = {0: [(1, (0, 0, 10, 10))], 1: [(1, (10, 0, 20, 10))]}
74         assert ff.mean_player_motion(per_frame, 100, 100) == pytest.approx(0.10)
75         # single frame -> 0; no shared track id -> 0
76         assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))]}, 100, 100) == 0.0

```

```

76         assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))], 1: [(2, (10, 0, 20, 10))]},
100, 100) == 0.0
77
78
79     # ----- two_sided_motion
80
81     def test_two_sided_motion():
82         net = ("y", 0.5)
83         near = (10, 10, 30, 40)      # foot y = 0.4 < 0.5 (near side)
84         far = (10, 60, 30, 90)      # foot y = 0.9 >= 0.5 (far side)
85         both = {0: [(1, near), (2, far)], 1: [(1, near)]}      # frame 0 two-sided, frame 1
one-sided
86         assert ff.two_sided_motion(both, 100, 100, net, None) == pytest.approx(0.5)
87         one_sided = {0: [(1, near)], 1: [(2, near)]}
88         assert ff.two_sided_motion(one_sided, 100, 100, net, None) == 0.0
89         assert ff.two_sided_motion(both, 100, 100, None, None) == 0.0      # no net -> 0
90
91
92     # ----- shuttle_player_colocation
93
94     def test_shuttle_player_colocation():
95         box = (40, 40, 60, 60)
96         per_frame = {0: [(1, box)], 1: [(1, box)], 2: [(1, box)]}
97         # frame0 shuttle inside box (held), frame1 shuttle far (in-flight), frame2
invisible.
98         shuttle = [_shuttle(0, 50, 50, True), _shuttle(1, 95, 95, True), _shuttle(2, 50, 50,
False)]
99         # aligned visible frames = {0, 1}; colocated = {0} -> 0.5
100         assert ff.shuttle_player_colocation(per_frame, shuttle, 100, 100) ==
pytest.approx(0.5)
101         # all-held -> 1.0
102         held = [_shuttle(0, 50, 50, True), _shuttle(1, 50, 50, True)]
103         assert ff.shuttle_player_colocation(per_frame, held, 100, 100) == pytest.approx(1.0)
104         # no shuttle -> 0.0; shuttle frame not person-sampled -> skipped (no aligned -> 0.0)
105         assert ff.shuttle_player_colocation(per_frame, [], 100, 100) == 0.0
106         assert ff.shuttle_player_colocation(per_frame, [_shuttle(99, 50, 50, True)], 100,
100) == 0.0
107
108
109     # ----- aggregator
110
111     def test_compute_person_features_keys():
112         near = (10, 10, 30, 40)
113         far = (10, 60, 30, 90)
114         per_frame = {0: [(1, near), (2, far)], 1: [(1, near), (2, far)]}
115         shuttle = [_shuttle(0, 20, 25, True)]
116         feats = ff.compute_person_features(per_frame, shuttle, 100, 100,
court_polygon=None, net=("y", 0.5))
117
118         assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
119         assert all(isinstance(v, float) for v in feats.values())
120         assert feats["two_sided_motion"] == pytest.approx(1.0)      # both sides occupied every
frame
121
122
123     def test_compute_person_features_empty():
124         feats = ff.compute_person_features({}, [], 100, 100)
125         assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
126         assert all(v == 0.0 for v in feats.values())
127

```

20. assistant (2026-06-13T19:10:06.075Z)

Now I have enough to verify the alignment claims rigorously. Let me focus on the key concern: the frame-skip sampling alignment between `detections\_per\_frame` (player sampling) and the shuttle trajectory `.frame` keys used by `shuttle\_player\_colocation`.

Let me trace through the critical alignment logic:

1. ****Shuttle trajectory**** (``parse_trajectory_csv``): produces ``TrajectoryPoint`` with ``.frame`` = every frame index from the CSV (frame 0, 1, 2, 3, ... ? WASB processes every frame).
2. ****Player sampling**** (``detections_per_frame``): samples frames where ``(frame_idx - start_frame) % step == 0``. So it samples ``start_frame, start_frame+step, start_frame+2*step, ...``.
3. ****`shuttle\_player\_colocation`****: for each **visible** shuttle point ``p``, looks up ``per_frame.get(int(p.frame))``. If ``None`` (frame not person-sampled), it's skipped (``continue``). Only frames where BOTH a visible shuttle exists AND the frame was player-sampled count toward ``aligned``.

Let me verify the frame-skip arithmetic concern precisely and check the float round-trip in the fusion segmenter.

21. user (2026-06-13T19:10:09.502Z)

```
frame=30 start_sec=1.0 -> int(start_sec*fps)=30 match=True
frame=31 start_sec=1.0333333333333334 -> int(start_sec*fps)=31 match=True
frame=89 start_sec=2.9666666666666667 -> int(start_sec*fps)=89 match=True
frame=119 start_sec=3.9666666666666667 -> int(start_sec*fps)=119 match=True
frame=300 start_sec=10.0 -> int(start_sec*fps)=300 match=True
frame=359 start_sec=11.966666666666667 -> int(start_sec*fps)=359 match=True
```

22. assistant (2026-06-13T19:10:13.314Z)

The round-trip is generally exact, but ``int()`` truncation toward zero is a known floating-point hazard. Let me probe for cases where ``int(frame/fps*fps)`` lands on ``frame-1`` due to floating point representation (a classic bug for frame-to-time-to-frame round trips), and check what happens with non-integer fps like 29.97.

23. user (2026-06-13T19:10:16.051Z)

```
fps=30.0: 404 mismatches [(123, 122), (245, 244), (246, 245), (247, 246), (490, 489), (492, 491)]
fps=29.97: 477 mismatches [(61, 60), (122, 121), (241, 240), (244, 243), (247, 246), (482, 481)]
fps=25.0: 1145 mismatches [(29, 28), (57, 56), (58, 57), (113, 112), (114, 113), (115, 114)]
fps=59.94: 477 mismatches [(61, 60), (122, 121), (241, 240), (244, 243), (247, 246), (482, 481)]
fps=50.0: 1145 mismatches [(29, 28), (57, 56), (58, 57), (113, 112), (114, 113), (115, 114)]
fps=24.0: 0 mismatches []
```

24. assistant (2026-06-13T19:10:24.922Z)

This confirms a real off-by-one hazard at the window edge: ``int(cw["start"] * fps)`` can land on ``frame - 1`` for hundreds of frames at fps=30. But this is the **window start**, which only shifts the sampling phase by one frame ? let me assess whether that actually breaks colocation alignment. The more important alignment question is the ****sampling phase consistency****: does ``detections_per_frame``'s set of sampled frame indices coincide with the shuttle ``.frame`` values?

Let me trace the actual alignment mechanism carefully. The key insight: ``shuttle_player_colocation`` aligns by exact frame-index dict lookup ``per_frame.get(int(p.frame))``. Both the shuttle ``.frame`` and the player-sampled ``frame_idx`` are absolute original-video frame indices, so they **can** coincide ? but only at frames where ``(frame_idx - start_frame) % step == 0``. Let me quantify the alignment rate.

25. user (2026-06-13T19:10:28.308Z)

```
exact start: start_f=30 sampled=30 aligned_with_shuttle=30 (rate=0.33)
off-by-one start (truncation): start_f=29 sampled=31 aligned_with_shuttle=30 (rate=0.33)
```

26. assistant (2026-06-13T19:10:35.885Z)

So the alignment mechanism itself works: $\sim 1/\text{step}$ of shuttle frames align with player-sampled

frames, and `shuttle\_player\_colocation` correctly only counts aligned frames. The off-by-one start shift doesn't break alignment (it just shifts which subset of frames are sampled). Good ? the docstring claim that "returned frame indices match the WASB trajectory's `.frame` exactly" is accurate for the sampled subset.

Now let me verify the most important concern about `cap.set(CAP\_PROP\_POS\_FRAMES)` + sequential read frame labeling. The code does `cap.set(POS\_FRAMES, start\_frame)` then labels the first decoded frame as `frame\_idx = start\_frame`. This is the correctness claim. Let me also check the `end\_frame` boundary and the resource-release on all paths.

Let me check one subtle bug: the `while frame\_idx <= end\_frame` loop combined with `cap.set` seek. Let me also examine whether `lazy\_load\_model` is called but the early-return path (cap not opened) still works, and the tracker fps.

Let me look closely at the tracker fps consistency between the two methods and verify the `\_person\_features` window extraction has no off-by-one with `end\_f`.

27. user (2026-06-13T19:10:39.220Z)

tracker fps convention consistent: 10.0 == 10.0

28. assistant (2026-06-13T19:10:43.015Z)

Now let me examine the critical resource-release and seek-correctness concerns in detail, plus check whether `end\_frame` could be `< start\_frame` (empty/degenerate window) and the `det["frame\_height"]` or 0.0` fallback in `_person_features`.

Let me re-read the key lines and check the `\_person\_features` fallback logic for `fh`:

29. user (2026-06-13T19:10:44.959Z)

```
85         def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                               frame_width: float, video_path: str,
87                               fcfg: Dict[str, Any]) -> Dict[str, float]:
88             """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89             frame range and compute the scale/translation-invariant person features. Lazily
builds
90             ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91             # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92             from backend.pipeline.detectors.person_detector import PersonDetector
93             if self._person is None:
94                 self._person = PersonDetector(self.config)
95
96             frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97             start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98             det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99             fw = det["frame_width"] or frame_width
100            fh = det["frame_height"] or 0.0
101
102            poly = fcfg.get("court_polygon")
103            polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104            net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105            shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106            return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                             court_polygon=polygon, net=net)
```

30. assistant (2026-06-13T19:10:54.955Z)

Now I've found a meaningful concern. Let me verify it: in `\_person\_features`, `fh = det["frame\_height"] or 0.0`. If `detections\_per\_frame` returns `frame\_height = 0.0` (which happens when `cap.get(CAP\_PROP\_FRAME\_HEIGHT)` fails ? the early-return path returns `0.0`, and

the normal path doesn't guard against ` $\leq 0$ `), then ` $fh = 0.0$ '. With ` $fh = 0.0$ ', all the person features that depend on ``frame_height`` (`person_in_court`, ``_foot_norm`', ``two_sided_motion`', ``mean_player_motion`', ``shuttle_player_colocation`') degrade to 0.0. Let me confirm ``detections_per_frame`' doesn't guard ``frame_height`/`frame_width`' against ` ≤ 0 ' like the other methods do.

Looking at lines 282-283 of `person_detector.py`:

```
```python
frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
```
```

Neither is guarded against ` ≤ 0 ' (unlike ``fps`' on line 279-281, and unlike ``extract_action_windows_from_chunk`' which guards ``frame_width`'). This is asymmetric but the consumer ``_person_features`' partially compensates for ``frame_width`' (`fw = det["frame_width"]` or `frame_width`) but NOT for ``frame_height`' (`fh = det["frame_height"]` or 0.0). Let me verify whether there's a real path where this matters, and check the WASB coordinate space concern (shuttle x/y pixel space vs the frame dims used for colocation).

Let me check the WASB infer output coordinate space to confirm shuttle x,y are in the same pixel space as the person detector frame dims (critical for ``shuttle_player_colocation`' and ``_shuttle_in_box`').

31. user (2026-06-13T19:10:55.150Z)

```
1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's tracknet_runner.parse_trajectory_csv already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL wasb conda env, from the WASB-SBDT src/ dir (it
11     imports WASB's detectors/trackers/dataloaders). The Windows-side adapter
12     (wasb_runner.py) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       <cache>/det_raw.jsonl (flushed+fsync'd every --flush-every windows) and
23       records progress in <cache>/manifest.json (atomic write). A reboot loses at
24       most --flush-every windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (<out>_wasbcache/ by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36         python wasb_infer.py --frames_dir /path/frames --weights
37         ../pretrained_weights/wasb_badminton_best.pth.tar \
38         --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39         every 1000] [--fresh]
40     """
```

```

39     import os
40     import os.path as osp
41     import csv
42     import glob
43     import json
44     import argparse
45     import logging
46     from collections import defaultdict
47
48     logger = logging.getLogger(__name__)
49
50     CACHE_VERSION = 1
51     _DET_FILE = "det_raw.jsonl"
52     _MANIFEST_FILE = "manifest.json"
53
54
55     def _build_cfg(weights: str, sport: str):
56         """Compose the WASB eval config via Hydra, overriding model/weights/device."""
57         from hydra import compose, initialize
58         with initialize(config_path="configs", version_base=None):
59             cfg = compose(config_name="eval", overrides=[
60                 f"dataset={sport}",
61                 "model=wasb",
62                 f"detector.model_path={weights}",
63                 "runner.device=cuda",
64                 "runner.gpus=[0]",          # this box has a single GPU; default config assumes
more
65             ])
66         return cfg
67
68
69     def _frame_id(path: str) -> int:
70         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
71         stem = osp.splitext(osp.basename(path))[0]
72         digits = "".join(ch for ch in stem if ch.isdigit())
73         return int(digits) if digits else -1
74
75
76     # ----- #
77     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
78     # ----- #
79     def _cache_paths(cache_dir: str):
80         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
81
82
83     def default_cache_dir(out_csv: str) -> str:
84         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
85         d = osp.dirname(osp.abspath(out_csv))
86         stem = osp.splitext(osp.basename(out_csv))[0]
87         return osp.join(d, stem + "_wasbcache")
88
89
90     def _atomic_write_text(path: str, text: str) -> None:
91         """Write then rename so a crash mid-write can't leave a half-written file."""
92         tmp = path + ".tmp"
93         with open(tmp, "w") as f:
94             f.write(text)
95             f.flush()
96             os.fsync(f.fileno())
97         os.replace(tmp, path)
98
99
100     def _det_plain(det) -> list:
101         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able

```

```

list."""
102     xy = det["xy"]
103     return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
104
105
106 def _dets_to_tracker(plain_list):
107     """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
108     the tracker does vector math / np.linalg.norm on it)."""
109     import numpy as np
110     return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
111             for p in plain_list]
112
113
114 def write_manifest(cache_dir: str, manifest: dict) -> None:
115     _, mpath = _cache_paths(cache_dir)
116     _atomic_write_text(mpath, json.dumps(manifest, indent=2))
117
118
119 def read_manifest(cache_dir: str):
120     _, mpath = _cache_paths(cache_dir)
121     if not osp.exists(mpath):
122         return None
123     try:
124         with open(mpath) as f:
125             return json.load(f)
126     except (json.JSONDecodeError, OSError):
127         return None
128
129
130 def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
131                         frames_in: int, frames_total: int) -> bool:
132     """A cache is reusable only if the run that produced it matches this run's
inputs."""
133     if not manifest or manifest.get("version") != CACHE_VERSION:
134         return False
135     return (
136         manifest.get("frames_dir") == osp.abspath(frames_dir)
137         and manifest.get("weights") == weights
138         and manifest.get("sport") == sport
139         and manifest.get("frames_in") == frames_in
140         and manifest.get("frames_total") == frames_total
141     )
142
143
144 def reset_cache(cache_dir: str) -> None:
145     """Drop any existing cache so the next run starts clean."""
146     det_jsonl, mpath = _cache_paths(cache_dir)
147     for p in (det_jsonl, mpath):
148         if osp.exists(p):
149             os.remove(p)
150
151
152 def load_and_clean_cache(cache_dir: str):
153     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
154     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
155     prefix so a trailing half-written line from a crash can't corrupt future appends.
156
157     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
158     [x, y, score, scale] candidates (accumulated across every window the frame is in,
159     in window order ? identical to a non-resumed run).
160     """
161     det_jsonl, _ = _cache_paths(cache_dir)
162     det_by_fid = defaultdict(list)
163     valid: list = []

```

```

164         if osp.exists(det_jsonl):
165             with open(det_jsonl, "r") as f:
166                 for line in f:
167                     s = line.strip()
168                     if not s:
169                         continue
170                     try:
171                         obj = json.loads(s)
172                     except json.JSONDecodeError:
173                         break # truncated trailing line (crash mid-
flush)
174                     if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
175                         break
176                     valid.append(s)
177                     for fid, plain in obj["f"]:
178                         det_by_fid[int(fid)].extend([list(p) for p in plain])
179             # Guarantee a clean append boundary by rewriting only the good prefix.
180             _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
181         return len(valid), det_by_fid
182
183
184 def _append_windows(fh, objs) -> None:
185     """Append window records as JSONL and durably flush (bounded loss on crash)."""
186     for o in objs:
187         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
188     fh.flush()
189     os.fsync(fh.fileno())
190
191
192 def _atomic_write_trajectory(out_csv: str, rows) -> None:
193     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
194     tmp = out_csv + ".tmp"
195     with open(tmp, "w", newline="") as f:
196         w = csv.writer(f)
197         w.writerow(["Frame", "Visibility", "X", "Y"])
198         w.writerows(rows)
199         f.flush()
200     os.fsync(f.fileno())
201     os.replace(tmp, out_csv)
202
203
204 # ----- #
205 # Inference
206 # ----- #
207 def run(frames_dir: str, weights: str, out_csv: str, sport: str = "badminton",
208         limit: int = 0, batch_size: int = 8, num_workers: int = 4,
209         log_every_batches: int = 50, cache_dir: "str | None" = None,
210         flush_every: int = 1000, fresh: bool = False) -> str:
211     import time
212     import torch
213     from torch.utils.data import DataLoader
214     from detectors import build_detector
215     from trackers import build_tracker
216     from dataloaders import build_img_transforms, build_seq_transforms
217     from dataloaders.dataset_loader import ImageDataset
218     from utils import Center
219
220     cfg = _build_cfg(weights, sport)
221     frames_in = int(cfg["model"]["frames_in"])
222     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
223     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
224
225     frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
glob.glob(osp.join(frames_dir, "*.jpg")))
226     if limit:

```



```

227         frames = frames[:limit]
228     if len(frames) < frames_in:
229         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in
{frames_dir}")
230
231     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
232     samples = []
233     for i in range(len(frames) - frames_in + 1):
234         window = frames[i:i + frames_in]
235         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
236         samples.append({"frames": window, "annos": annos})
237     windows_total = len(samples)
238
239     # ---- cache setup / resume decision ----- #
240     if cache_dir is None:
241         cache_dir = default_cache_dir(out_csv)
242         os.makedirs(cache_dir, exist_ok=True)
243         det_jsonl, _ = _cache_paths(cache_dir)
244         manifest = None if fresh else read_manifest(cache_dir)
245         resume = manifest_compatible(
246             manifest or {}, frames_dir=frames_dir, weights=weights, sport=sport,
247             frames_in=frames_in, frames_total=len(frames),
248         )
249         if resume:
250             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
251             logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
252         else:
253             if manifest is not None:
254                 logger.info("cache incompatible with this run -> starting fresh")
255                 reset_cache(cache_dir)
256                 windows_done, det_by_fid = 0, defaultdict(list)
257
258     base_manifest = {
259         "version": CACHE_VERSION,
260         "frames_dir": osp.abspath(frames_dir),
261         "weights": weights,
262         "sport": sport,
263         "frames_in": frames_in,
264         "frames_total": len(frames),
265         "windows_total": windows_total,
266         "windows_done": windows_done,
267     }
268     write_manifest(cache_dir, base_manifest)
269
270     # ---- detector pass over the un-cached windows ----- #
271     remaining = samples[windows_done:]
272     if remaining:
273         _, transform_test = build_img_transforms(cfg)
274         try:
275             _, seq_transform_test = build_seq_transforms(cfg)
276         except Exception:
277             seq_transform_test = None
278         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
279                         transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
280         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=num_workers)
281         detector = build_detector(cfg)
282
283         t0 = time.time()
284         done = windows_done
285         win_idx = windows_done

```

```

286         log_every = max(1, log_every_batches)
287         flush_n = max(1, flush_every)
288         pending = []
289         logger.info(f"detecting on {len(remaining)} remaining windows ")
290         f"(total {windows_total}, batch={batch_size},
workers={num_workers})..."
291         det_f = open(det_jsonl, "a")
292         try:
293             with torch.no_grad():
294                 for bi, batch in enumerate(loader):
295                     imgs, _hms, trans = batch[0], batch[1], batch[2]
296                     img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
297                     batch_results, _ = detector.run_tensor(imgs, trans)
298                     nb = imgs.shape[0]
299                     for ib in range(nb):
300                         frames_payload = []
301                         for ie in sorted(batch_results[ib].keys()):
302                             p = img_paths[ie][ib]
303                             fid = _frame_id(p)
304                             plain = [_det_plain(d) for d in batch_results[ib][ie]]
305                             frames_payload.append([fid, plain])
306                             det_by_fid[fid].extend(plain)
307                         pending.append({"w": win_idx, "f": frames_payload})
308                         win_idx += 1
309                     done += nb
310                     if len(pending) >= flush_n or done >= windows_total:
311                         _append_windows(det_f, pending)
312                         pending = []
313                         base_manifest["windows_done"] = done
314                         write_manifest(cache_dir, base_manifest)
315                     if (bi + 1) % log_every == 0 or done >= windows_total:
316                         el = time.time() - t0
317                         new_done = done - windows_done
318                         rate = new_done / el if el > 0 else 0.0
319                         eta = (windows_total - done) / rate if rate > 0 else 0.0
320                         logger.info(f"{done}/{windows_total} "
321                                     f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
322                                     f"elapsed {el:.0f}s | ETA {eta:.0f}s")
323                     if pending:
324                         _append_windows(det_f, pending)
325                         base_manifest["windows_done"] = done
326                         write_manifest(cache_dir, base_manifest)
327                 finally:
328                     det_f.close()
329                 logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
330             else:
331                 logger.info("detector cache already complete -> tracker-only re-run")
332
333             # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
334             tracker = build_tracker(cfg)
335             tracker.refresh()
336             rows = []
337             for fid in sorted(det_by_fid.keys()):
338                 r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
339                 rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
340
341             _atomic_write_trajectory(out_csv, rows)
342             visible = sum(1 for _, v, *_ in rows if v)
343             logger.info(f"wrote {len(rows)} rows -> {out_csv}")
344             logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
345             return out_csv
346
347

```

```

348 def extract_frames(video_path: str, frames_dir: str) -> str:
349     """Decode a video to PNG frames named by source frame index (00000000.png ...).
350
351     Idempotent: if the dir already holds the expected number of frames (matching the
352     container's reported frame count) we skip re-decoding ? re-extraction after a
353     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
354     extracting with ffmpeg on the host and passing --frames_dir.
355     """
356     import cv2
357     os.makedirs(frames_dir, exist_ok=True)
358     cap = cv2.VideoCapture(video_path)
359     if not cap.isOpened():
360         raise SystemExit(f"cannot open video: {video_path}")
361     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
362     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
363     if expected > 0 and existing >= expected:
364         cap.release()
365         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
366         return frames_dir
367     i = 0
368     while True:
369         ok, frame = cap.read()
370         if not ok:
371             break
372         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
373         i += 1
374     cap.release()
375     logger.info(f"extracted {i} frames -> {frames_dir}")
376     return frames_dir
377
378
379 def main():
380     ap = argparse.ArgumentParser()
381     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
382     ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
383     ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
384     ap.add_argument("--weights", required=True)
385     ap.add_argument("--out", required=True)
386     ap.add_argument("--sport", default="badminton")
387     ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
388     ap.add_argument("--batch-size", type=int, default=8)
389     ap.add_argument("--num-workers", type=int, default=4)
390     ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
391     ap.add_argument("--cache-dir", default=None,
392                     help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
393     ap.add_argument("--flush-every", type=int, default=1000,
394                     help="fsync the detector cache every N windows (bounds reboot
loss)")
395     ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
396     args = ap.parse_args()
397
398     frames_dir = args.frames_dir
399     if args.video:
400         frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
401     if not frames_dir:
402         raise SystemExit("provide --frames_dir or --video")
403     run(frames_dir, args.weights, args.out, args.sport, args.limit,
404         batch_size=args.batch_size, num_workers=args.num_workers,

```

```

405         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
406         flush_every=args.flush_every, fresh=args.fresh)
407
408
409     if __name__ == "__main__":
410         main()
411

```

32. assistant (2026-06-13T19:11:06.232Z)

This is the critical finding. The WASB coordinate space is in the **model's output resolution**, NOT the original video pixel resolution. Let me verify this carefully.

Looking at ``wasb_infer.py``:

```

- Line 223: `output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))`
- Line 339: `rows.append((fid, ..., r.get("x", -1), r.get("y", -1)))` ? the tracker's x/y.

```

The detector runs on resized input (``input_wh``) and produces heatmaps at ``output_wh``. The tracker x/y coordinates are in the WASB model's coordinate space (output heatmap resolution, e.g. 512×288), **not** the original video's pixel resolution.

Meanwhile, ``detections_per_frame`` returns ``frame_width`/`frame_height`` = the **original video** resolution (e.g. 1920×1080 or 3840×2160). The person bounding boxes are in original-video pixels.

```

`shuttle_player_colocation` (`_shuttle_in_box`) compares the shuttle `(sx, sy)` against person box `(x1,y1,x2,y2)` directly in pixels:
```python
return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
```

```

If the shuttle x/y is in WASB output-resolution space (~512×288) but the person box is in original-video pixels (~1920×1080), these are in **different coordinate spaces** and the colocation test will essentially never be true (shuttle coords would be in the top-left corner relative to the person boxes). This would silently make ``shuttle_player_colocation`` ? 0 always.

Let me verify what coordinate space the WASB tracker actually emits by checking the WASB-SBDT online tracker / how ``r.get("x")`` relates to input/output resolution. Let me check if there's any rescaling in the runner or detector path, and look for documentation of the trajectory coordinate space.

33. user (2026-06-13T19:11:08.852Z)

```

backend\pipeline\detectors\fusion_features.py:6:module only turns those signals into a handful
of scale/translation-invariant
backend\pipeline\detectors\fusion_features.py:7:features (counts / ratios / zone-membership ?
never raw pixel coordinates, which would
backend\pipeline\detectors\fusion_features.py:31:     """Ray-casting point-in-polygon. `point`
and `polygon` share a coordinate space
backend\pipeline\detectors\fusion_features.py:51:def _foot_norm(box: BBox, frame_width: float,
frame_height: float) -> Tuple[float, float]:
backend\pipeline\detectors\fusion_features.py:54:     if frame_width <= 0 or frame_height <= 0:
backend\pipeline\detectors\fusion_features.py:57:     return ((x1 + x2) / 2.0 / frame_width, y2 /
frame_height)
backend\pipeline\detectors\fusion_features.py:60:def _center_norm(box: BBox, frame_width: float,
frame_height: float) -> Tuple[float, float]:
backend\pipeline\detectors\fusion_features.py:61:     if frame_width <= 0 or frame_height <= 0:
backend\pipeline\detectors\fusion_features.py:64:     return ((x1 + x2) / 2.0 / frame_width, (y1
+ y2) / 2.0 / frame_height)
backend\pipeline\detectors\fusion_features.py:67:def person_in_court(box: BBox, frame_width:
float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:73:     return point_in_polygon(_foot_norm(box,
frame_width, frame_height), court_polygon)
backend\pipeline\detectors\fusion_features.py:76:def in_court_counts(per_frame: Dict[int,
List[Tuple[int, BBox]]], frame_width: float,

```

```

backend\pipeline\detectors\fusion_features.py:81:         sum(1 for _tid, box in dets if
person_in_court(box, frame_width, frame_height, court_polygon))
backend\pipeline\detectors\fusion_features.py:107:         frame_width: float,
frame_height: float) -> float:
backend\pipeline\detectors\fusion_features.py:109:         by the frame width (resolution-invariant).
0.0 if <2 frames or no shared tracks."""
backend\pipeline\detectors\fusion_features.py:110:         if frame_width <= 0 or len(per_frame) < 2:
backend\pipeline\detectors\fusion_features.py:118:         cx0, cy0 =
_center_norm(prev[tid], frame_width, frame_height)
backend\pipeline\detectors\fusion_features.py:119:         cx1, cy1 = _center_norm(box,
frame_width, frame_height)
backend\pipeline\detectors\fusion_features.py:126: def two_sided_motion(per_frame: Dict[int,
List[Tuple[int, BBox]]], frame_width: float,
backend\pipeline\detectors\fusion_features.py:141:         if not person_in_court(box,
frame_width, frame_height, court_polygon):
backend\pipeline\detectors\fusion_features.py:143:         fx, fy = _foot_norm(box,
frame_width, frame_height)
backend\pipeline\detectors\fusion_features.py:154: def _shuttle_in_box(sx: float, sy: float, box:
BBox, frame_width: float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:159:         px = pad_frac * frame_width
backend\pipeline\detectors\fusion_features.py:165:         frame_width:
float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:171:         if not per_frame or frame_width <= 0 or
frame_height <= 0:
backend\pipeline\detectors\fusion_features.py:182:         if any(_shuttle_in_box(p.x, p.y, box,
frame_width, frame_height, pad_frac)
backend\pipeline\detectors\fusion_features.py:197:         frame_width:
float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:203:         counts = in_court_counts(per_frame,
frame_width, frame_height, court_polygon)
backend\pipeline\detectors\fusion_features.py:206:         "two_sided_motion":
two_sided_motion(per_frame, frame_width, frame_height, net, court_polygon),
backend\pipeline\detectors\fusion_features.py:207:         "mean_player_motion":
mean_player_motion(per_frame, frame_width, frame_height),
backend\pipeline\detectors\fusion_features.py:210:         per_frame, shuttle_points,
frame_width, frame_height),
backend\pipeline\detectors\person_detector.py:162:         frame_width =
cap.get(cv2.CAP_PROP_FRAME_WIDTH)
backend\pipeline\detectors\person_detector.py:163:         if frame_width <= 0:
backend\pipeline\detectors\person_detector.py:164:         frame_width = 1280.0 # Sensible
fallback
backend\pipeline\detectors\person_detector.py:186:         # gives ~10 samples/second, plenty
for human-scale velocity.
backend\pipeline\detectors\person_detector.py:203:         velocity =
get_velocity_normalised(prev_bbox, bbox, effective_fps, frame_width)
backend\pipeline\detectors\person_detector.py:268:         Returns ``{"frames": {frame_idx:
[(track_id, bbox), ...], "fps", "frame_width",
backend\pipeline\detectors\person_detector.py:277:         return {"frames": {}, "fps": 30.0,
"frame_width": 0.0, "frame_height": 0.0}
backend\pipeline\detectors\person_detector.py:282:         frame_width =
cap.get(cv2.CAP_PROP_FRAME_WIDTH)
backend\pipeline\detectors\person_detector.py:301:         "frame_width":
float(frame_width), "frame_height": float(frame_height)}

```

[Showing results with pagination = limit: 40]

34. assistant (2026-06-13T19:11:16.492Z)

The coordinate-space concern is real but it's a property of `fusion\_features` + the WASB trajectory, and confirming it definitively requires knowing the exact WASB tracker output scaling (which lives in the WASB-SBDT repo in WSL, not in this repo). The task scope is specifically the `PersonDetector.detections\_per\_frame` review, with the WASB-alignment question framed as "can the returned frame indices align with WASB trajectory `.frame`." The **frame-index** alignment (the temporal axis) is what `detections\_per\_frame` controls, and that is correct. The **spatial** coordinate-space mismatch is a `fusion\_features`/wiring concern ? I'll

note it as a real risk but flag it as needing the owner's WASB-side knowledge, not a definitively-confirmed bug testable here.

Let me now focus and confirm the genuinely testable logic bugs. Let me re-examine the `detections\_per\_frame` seek+read indexing for the precise correctness claim and the `end\_frame` handling. Let me run the actual test suite for the new files to confirm baseline green.

35. user (2026-06-13T19:11:26.600Z)

```
..... [100%]  
42 passed in 6.26s
```

36. assistant (2026-06-13T19:11:31.532Z)

All tests green. Now let me verify the specific logic-bug concerns testable without a GPU. Let me check the `detections\_per\_frame` behavior when `start\_frame > end\_frame` (degenerate window), and confirm the `cap.set` + sequential-read indexing claim. The key correctness property: after `cap.set(POS\_FRAMES, start\_frame)`, the first `cap.read()` returns the frame the code labels `start\_frame`.

Let me write a focused simulation of the loop logic (pure Python, no cv2) to confirm the sampling/labeling math, and check the boundary where `end\_frame` exceeds the video length vs the `ret` break.

37. user (2026-06-13T19:11:37.426Z)

```
normal: [30, 33, 36, 39]  
start>end (degenerate): []  
end past EOF: [995, 998]  
start clamped neg: [0, 3, 6]
```

38. assistant (2026-06-13T19:11:45.169Z)

The loop logic is sound: correct sampling, graceful EOF handling, degenerate window returns empty, negative start clamped. The frame-index labeling correctly assumes `cap.set(POS\_FRAMES, n)` makes the next `read()` return frame `n` ? which is the standard OpenCV contract (with the documented codec keyframe caveat noted in the docstring).

Now let me verify one more potential issue: the ****tracker continuity across the frame-skip gap****. In `detections\_per\_frame`, the tracker is only fed sampled frames (every `step`-th), but the loop reads (decodes) every frame in between and discards them. This is correct (it does NOT call `detect\_and\_track` on skipped frames). Good ? consistent with the docstring's tracker fps `fps/step`.

Let me double check there's no issue with `detect\_and\_track` being called with frames decoded sequentially but the tracker only seeing 1-in-step ? that matches `extract\_action\_windows\_from\_chunk` exactly. Confirmed consistent.

Now let me verify the `frame\_width`/`frame\_height` guard asymmetry is a real consumer-visible issue by checking what happens in `\_person\_features` when height is 0.

39. user (2026-06-13T19:11:50.429Z)

With frame_height=0, person features collapse: foot/center=(0,0), colocation=0.0
width has a fallback (fw), height does not (fh stays 0) -> ASYMMETRIC + silent zeroing

40. assistant (2026-06-13T19:12:03.442Z)

I've now verified all the concerns. Let me do a final check on whether `lazy\_load\_model()` is correctly called before the early-return (it is, line 273, before `cv2.VideoCapture`), and confirm the resource-release on all paths.

In `detections\_per\_frame`:
- Line 273: `self.lazy\_load\_model()` ? loads model (could raise if torch missing, but that's

GPU-side; the caller `\_person\_features` only runs when person cue is ON).

- Line 274-277: early return if `cap` not opened ? but `cap` was created on line 274.
- **`cap.release()` is NOT called on this early-return path.** Since `cap.isOpened()` is False, `release()` is arguably unnecessary (no resource held), but it's an inconsistency: the normal path releases (line 299). This is a low-severity nit ? when `isOpened()` is False, OpenCV holds no decoder, so the missing release is harmless. The normal path and the `not ret` break both reach `cap.release()` on line 299. Resource release is correct on all meaningful paths.

Let me verify the `points` filter uses `p.frame` as an int comparison correctly with the `start\_f`/`end\_f` int bounds (already confirmed `frame` is int from `parse\_trajectory\_csv`).

I now have a complete picture. Let me compile findings, ranked by severity.

Summary of verification:

- **Frame indexing (seek + sequential read)**: CORRECT. `cap.set(POS\_FRAMES, start\_frame)` then labeling the first read as `start\_frame` is the standard OpenCV contract; loop labels match decoded frames. Codec keyframe caveat is documented.
- **frame_skip sampling**: CORRECT. `(frame\_idx - start\_frame) % step == 0` samples `start\_frame, start\_frame+step, ...`; every intermediate frame is decoded but not run through the detector. Matches `extract\_action\_windows\_from\_chunk` convention.
- **Tracker fps**: CORRECT & consistent ? `make\_tracker(fps/step)` matches `effective\_fps` convention in the sibling method.
- **Resource release**: cap.release() on normal exit and EOF break. Missing on the `not isOpened()` early return, but harmless (no decoder held). Low.
- **WASB `.frame` alignment (temporal)**: CORRECT. Sampled absolute frame indices coincide with shuttle `.frame`; `shuttle\_player\_colocation` only counts aligned (person-sampled) visible shuttle frames.
- Real issues found below.

41. user (2026-06-13T19:12:36.579Z)

Structured output provided successfully

42. assistant (2026-06-13T19:12:48.671Z)

I reviewed `PersonDetector.detections\_per\_frame` and the surrounding fusion P2 wiring (`fusion\_features.py`, `fusion\_hybrid.py`, the `trajectory\_hybrid.py`/`models.py`/`\_\_init\_\_.py` edits), traced the WASB trajectory shape through `wasb\_runner.py`/`tracknet\_runner.py`/`wasb\_infer.py`, and ran the new test suite (42 passed).

Verdict on the specific items you asked about:

- **Seek + sequential-read frame indexing: CORRECT.** `cap.set(POS\_FRAMES, start\_frame)` then labeling the first `read()` as `start\_frame` and incrementing is the standard OpenCV contract; the loop labels match the decoded frames. The codec keyframe-tolerance caveat is honestly documented.
- **frame_skip sampling: CORRECT.** `(frame\_idx - start\_frame) % step == 0` samples `start\_frame, +step, +2\*step, ...`; intermediate frames are decoded but not run through the detector ? matches `extract\_action\_windows\_from\_chunk`.
- **Tracker fps: CORRECT & consistent.** `make\_tracker(fps/step)` mirrors the sibling method's `effective\_fps`.
- **Resource release: effectively correct.** Released on normal exit and EOF break; skipped only on the not-opened early return (harmless, no decoder held ? flagged low).
- **Temporal alignment with WASB `.frame`: CORRECT.** Sampled absolute indices coincide with shuttle `.frame`; `shuttle\_player\_colocation` only counts person-sampled visible shuttle frames.

The four findings (one high, one medium, two low) are in the StructuredOutput. The headline one is **not** in `detections\_per\_frame` itself but in how its output is fused: the WASB shuttle X/Y appear to live in the model's output-heatmap resolution while the person boxes and `frame\_width/height` are original-video pixels, so `shuttle\_player\_colocation` would silently read ~0 on real footage. That last claim needs owner-side confirmation of the WASB tracker's output scale (WSL/GPU side), which I flagged in the finding.

Key files: `backend/pipeline/detectors/person\_detector.py:256-301`,
`backend/pipeline/detectors/fusion\_features.py:154-185`,

```
`backend/pipeline/segmenters/fusion_hybrid.py:85-107`,  
`backend/pipeline/detectors/wasb_infer.py:223,337-341`.
```